

ENGR 692 GPU Computing

CUDA Implementation of Region Correlation Matrix Metrics

Abdul-Rashid Zakaria

December 15, 2025

1 Introduction

This report documents a standalone CUDA C++ implementation of Region Correlation Matrix (RCM) metrics for object based change detection. The implementation accelerates the computation of the region intersection matrix using GPU parallelism while preserving numerical parity with an existing Python reference implementation. The work targets segmentation outputs such as those produced by SAM based models and is intended for integration into larger change detection pipelines.

The core implementation, build instructions, and execution details are provided in the CUDA implementation notes. Aggregated experimental results, including tabular summaries and visualization plots, are discussed in the results documentation. A detailed code level walkthrough of the CUDA source file is given in the code walkthrough document.

The full CUDA implementation and supporting scripts are available on [GitHub](#). All results reported in this document were generated using the `cuda_rcm` branch.

2 Algorithm Overview

Given two segmentation masks A and B of identical spatial resolution, the RCM framework quantifies how regions in one mask correspond to regions in the other. The algorithm proceeds as follows:

1. Load both masks as grayscale label images, supporting 8 bit and 16 bit integer encodings.
2. Relabel arbitrary input labels to dense integer IDs $[0, K-1]$ to minimize memory footprint.
3. Construct an intersection matrix $M \in \mathbb{N}^{K_A \times K_B}$, where each entry counts pixel level overlap between region pairs.
4. Compute directional RCM metrics on the CPU using the dense intersection matrix.
5. Derive symmetric metrics by averaging forward and backward directional scores.

The computationally dominant step is the construction of the intersection matrix, which is parallelized on the GPU using a pixel parallel histogram style kernel.

3 CUDA Implementation

The main implementation is contained in `rcm_metrics_cuda.cu`. Each CUDA thread processes a single pixel and performs an atomic increment into the global intersection matrix. The flattened index is computed as

$$\text{idx} = a \cdot K_B + b,$$

where a and b are the relabeled region IDs from masks A and B .

Dense relabeling is performed on the CPU prior to GPU execution. This step mirrors the behavior of the Python `relabel_sequential` function and is essential to keep the intersection matrix compact and memory efficient.

Timing instrumentation distinguishes between kernel execution time, total GPU time including host device transfers, and a CPU intersection baseline. A rough throughput estimate in GFLOPs is computed assuming approximately three arithmetic operations per pixel, normalized by kernel execution time.

4 Metric Computation

Directional RCM metrics are computed on the CPU from the dense intersection matrix. For each region in the reference mask, the algorithm evaluates:

- **Overlap**, based on the maximum normalized intersection with any region in the comparison mask.
- **Fragmentation**, based on the number of nonzero overlapping parts and their relative contributions.
- **Composite score**, defined as the average of fragmentation and one minus overlap.

Symmetric metrics are obtained by computing directional scores in both directions, $A \rightarrow B$ and $B \rightarrow A$, and averaging the results. This ensures invariance to mask ordering and matches the behavior of the reference Python implementation.

5 Experimental Results

Aggregated results from multiple SAM based segmentation pairs demonstrate near perfect numerical agreement between the CUDA implementation and the Python reference. Metric differences are on the order of 10^{-6} , confirming correct relabeling and accumulation behavior.

Kernel execution times are consistently below a few milliseconds for the evaluated mask sizes. However, total GPU runtime is dominated by memory transfers and kernel launch overhead, resulting in end to end runtimes on the order of a few hundred milliseconds. For small masks, this overhead outweighs the raw kernel speedup, while larger scenes are expected to better amortize transfer costs.

5.1 Combined Metrics Visualization

Figure 1 visualizes symmetric overlap, fragmentation, composite scores, and timing statistics across multiple SAM based sites. Overlap values remain consistently high, indicating strong region correspondence between temporal masks. In contrast, fragmentation exhibits greater variability, reflecting differences in region splitting and merging behavior across scenes.

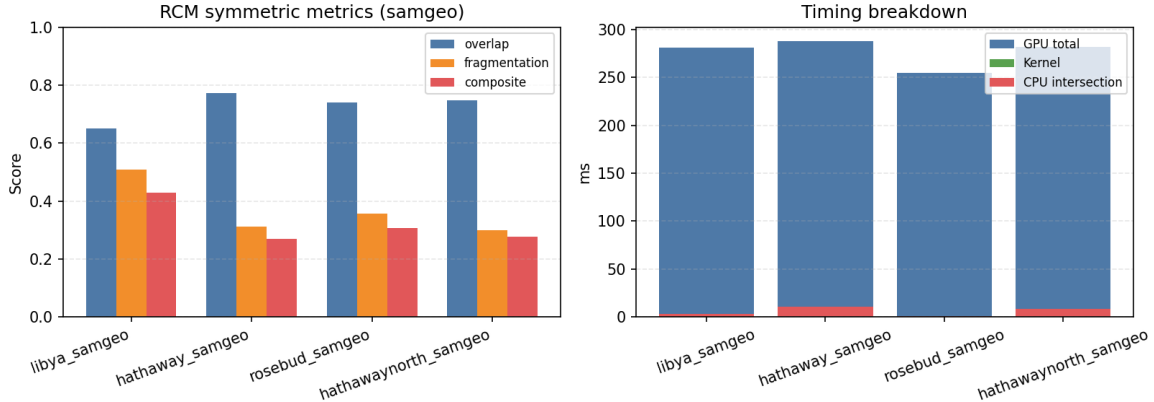


Figure 1: Combined symmetric RCM metrics and timing statistics across multiple SAM based segmentation sites. The figure compares overlap, fragmentation, composite scores, and execution times for the CUDA implementation.

The composite metric follows fragmentation trends more closely than overlap, consistent with its definition as an average of fragmentation and one minus overlap. As a result, scenes with similar overlap scores can still be distinguished by differences in composite scores. The timing bars further confirm that kernel execution time is negligible compared to total GPU runtime, emphasizing that data transfer overhead dominates for small to medium sized masks.

6 Limitations and Future Work

The current implementation allocates a dense $K_A \times K_B$ intersection matrix. While this is efficient for the evaluated datasets, extremely high region counts could lead to excessive memory usage. A sparse histogram approach or hash based accumulation strategy would address this limitation.

Additional improvements include overlapping host device transfers with computation using CUDA streams, exposing block size as a tunable parameter, and adding preflight checks to guard against excessive matrix allocations. Extending the implementation to support additional input formats such as NumPy arrays would further ease integration into Python based pipelines.

7 Conclusion

This report presented a CUDA accelerated implementation of Region Correlation Matrix metrics for object based change detection. The approach achieves exact numerical parity with a Python reference while significantly accelerating the intersection computation stage. The combined metrics visualization corroborates both numerical correctness and expected performance characteristics across multiple segmentation scenarios. The implementation is modular, self contained, and suitable for integration into larger GPU enabled change detection systems.